

DIC_Assignment 3

Habiba Moustafa

May 2025

1 Problem A

For the following Karnaugh map, give the circuit implementation using one 4-to-1 multiplexer and as many 2-to-1 multiplexers as required, but using as few as possible. You are not allowed to use any other logic gate and you must use *a* and *b* as the multiplexer selector inputs, as shown on the 4-to-1 multiplexer below.

Implementing just the portion labelled `top_module`, such that the entire circuit (including the 4-to-1 mux) implements the K-map.

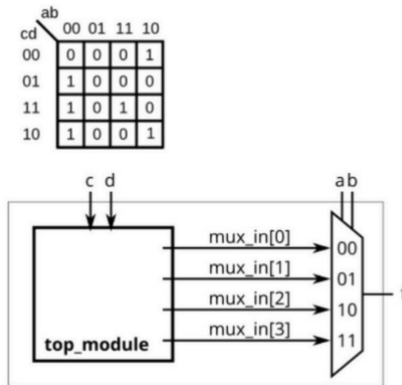


Figure 1: Problem A

1.1 Steps

- Design 2-to-1 Mux instance
- Design `top_module`

1.2 Implementation

- 2-to-1 MUX

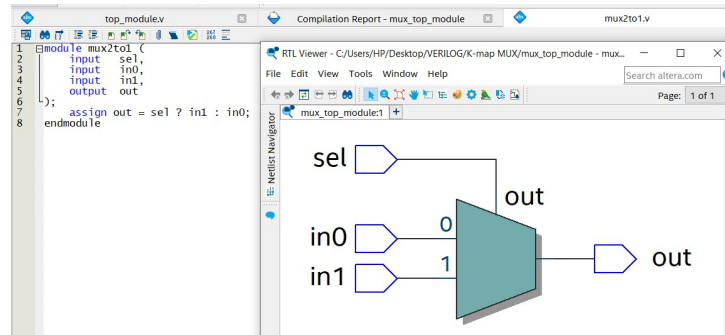


Figure 2: 2-to-1 MUX

- top_module

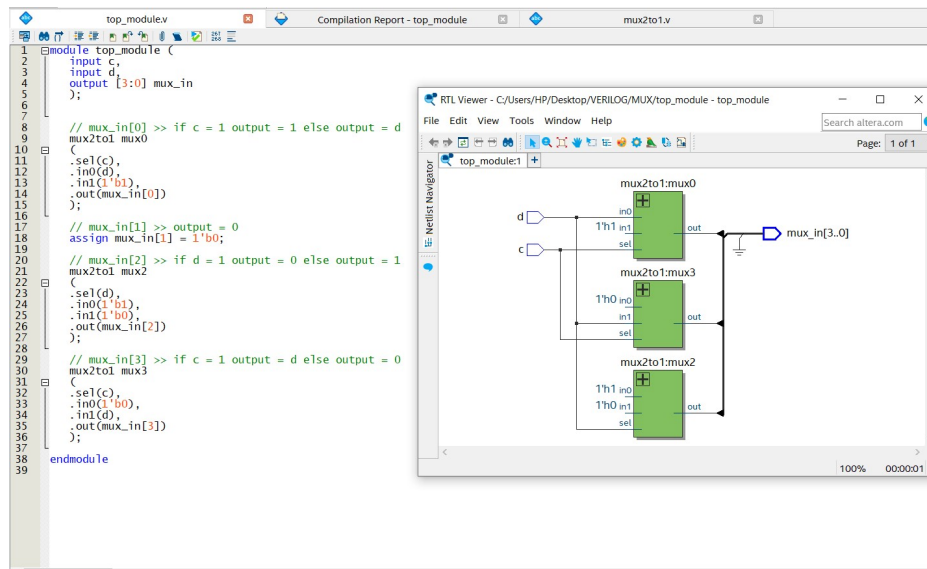


Figure 3: Top Module

1.3 Without using 2-to-1 MUX instances 1 **PROBLEM A**

1.3 Without using 2-to-1 MUX instances

```
module top_module (  
    input a,  
    input b,  
    input c,  
    input d,  
    output out  
);  
  
    wire [3:0] mux_in;  
  
    assign mux_in[0] = c ? 1'b1 : d; // if c output = 1 else output = d  
    assign mux_in[1] = 1'b0; // output = 0  
    assign mux_in[2] = d ? 1'b0 : 1'b1; // if d output = 0 else output = 1  
    assign mux_in[3] = c ? d : 1'b0; // if c output = d else output = 0  
  
    assign out = ({a, b} == 2'b00) ? mux_in[0] :  
                  ({a, b} == 2'b01) ? mux_in[1] :  
                  ({a, b} == 2'b10) ? mux_in[2] :  
                  mux_in[3];  
  
endmodule
```

Figure 4: Top Module 2

2 Problem B

Design a parameterized N-bit Ripple Carry Adder (RCA) using Verilog.

2.1 Requirements

- Use a parameter WIDTH = 4 to define the number of bits (N).
- Inputs:
 - a: N-bit input operand
 - b: N-bit input operand
 - cin: 1-bit input carry
- Outputs:
 - sum: N-bit sum output
 - cout: Final carry out

2.2 Implementation

Half Adder >> Full Adder >> N-bit Ripple Carry Adder

2.2.1 Half Adder

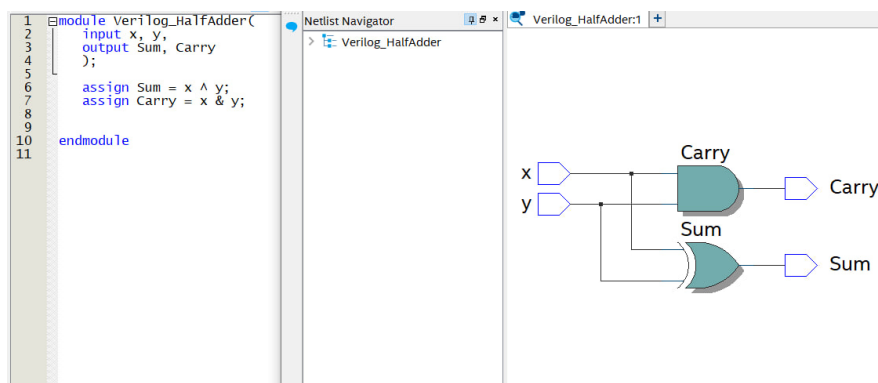


Figure 5: Half Adder

2.2.2 Full Adder

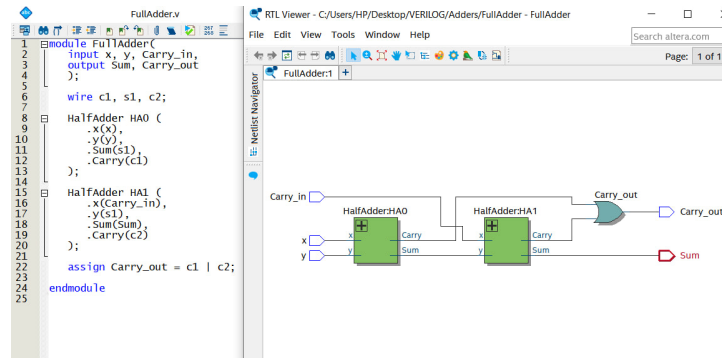


Figure 6: Full Adder

2.2.3 N-bit Ripple Carry Adder

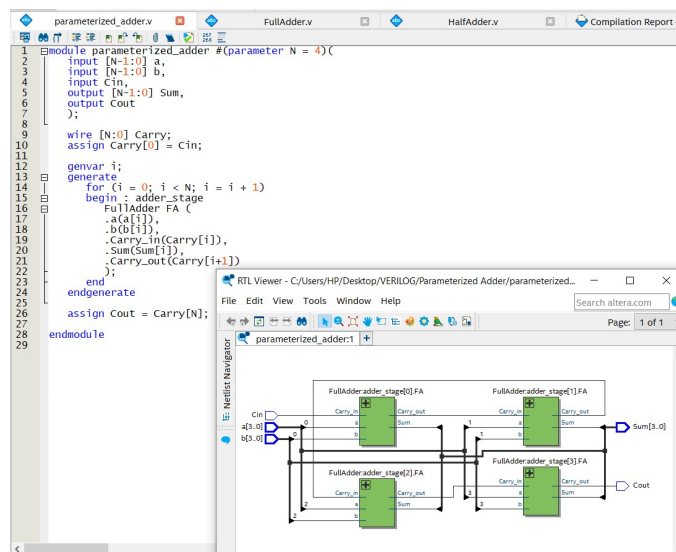


Figure 7: Parameterized Adder

3 Problem C

Create 8 D flip-flops with active high asynchronous reset.
All DFFs should be triggered by the positive edge of clk.

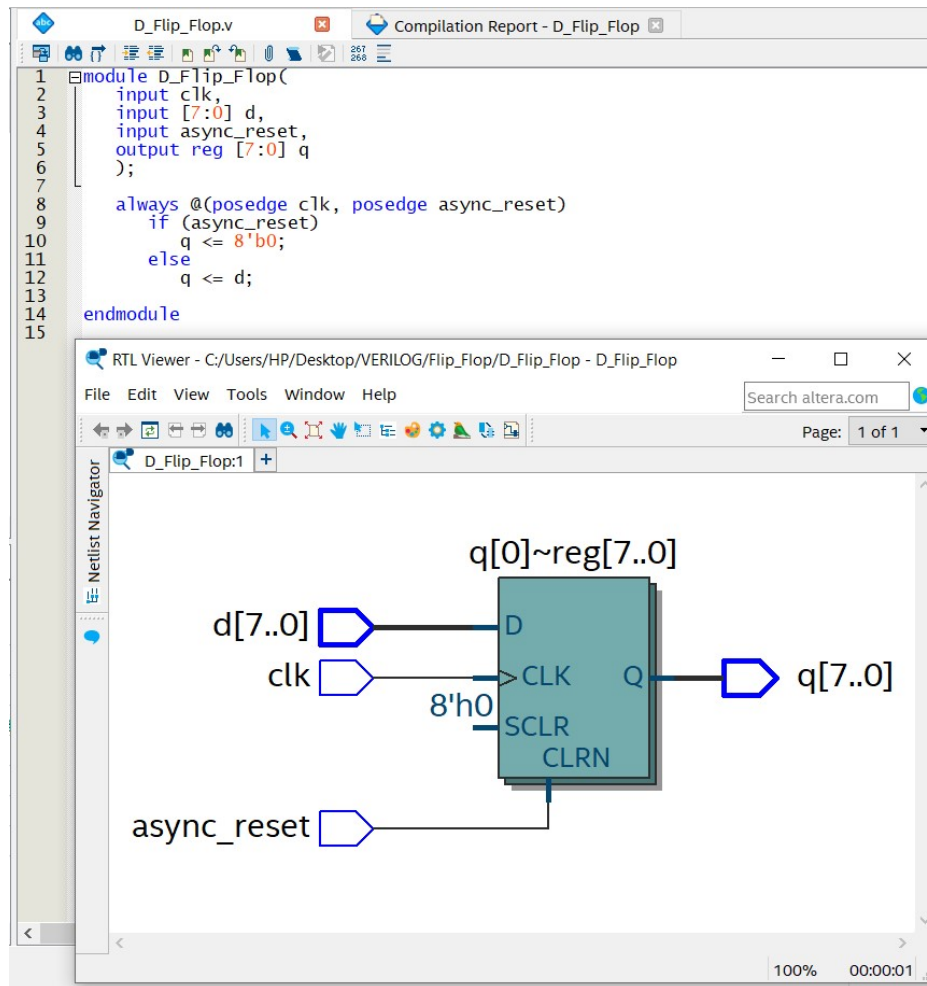


Figure 8: D_Flip_Flop